HDI Predictor: A Machine Learning Web App for Human Development Index Estimation

Project Description:

HDI Predictor is a Flask-based web application that estimates a country's Human Development Index (HDI) — the UNDP's composite measure of life expectancy, education, and income — and classifies it into an official development tier (Very High, High, Medium, or Low). The platform includes a **Home** dashboard summarizing global HDI statistics and exploratory data visuals, a **Predict** dashboard where a user manually enters four indicators to receive an instant prediction, and a **History** dashboard that logs and lets users review, filter, and manage past predictions.

The underlying model is a Linear Regression trained on real 2021 UNDP data across 191 countries, achieving an R^2 of 0.999 after log-transforming Gross National Income to mirror the income sub-index used in the UNDP's own HDI formula. Built with Flask and deployed publicly via Render, the platform ensures a seamless, no-installation experience for policymakers, researchers, and students exploring human development data.

Scenarios

Scenario 1: A user enters high life expectancy (82 years), strong schooling figures (17 expected / 13 mean years), and a high GNI per capita ($55,000). The system predicts a **Very High** HDI, demonstrating how strong performance across all three dimensions compounds into top-tier development.

Scenario 2: A policymaker enters mid-range values representing a developing nation. The system returns a **Medium** HDI, highlighting where targeted investment in health, education, or income could meaningfully shift the outcome.

Scenario 3: A researcher enters low life expectancy (52 years), limited schooling, and a low GNI ($1,200). The system predicts a **Low** HDI, flagging the profile as one requiring development intervention — exactly the kind of insight a development organization would use to prioritize investment.

Technical Architecture

HDI Predictor uses a modular three-layer architecture. The **presentation layer** (HTML, CSS, Chart.js) renders the three dashboards and communicates with the backend via a small JSON API. The **application layer** (Flask) routes requests to two isolated modules — model_utils.py for predictions and db_utils.py for history persistence — keeping the model and storage logic independently swappable. The **data layer** consists of a serialized scikit-learn model (model.pkl), a compact per-country reference dataset (country_data.json), and a SQLite database (predictions.db) for prediction history.

Note: this project uses a database (SQLite) for prediction history. See the Database Schema table in 3. Project Design Phase / Solution Architecture.pdf for the full table definition (columns, types, and indexing) in place of a separate ER diagram, since the schema is a single flat table.

Pre-requisites

- Flask Framework Knowledge: https://flask.palletsprojects.com/
- scikit-learn Familiarity: https://scikit-learn.org/stable/
- HTML, CSS, and JavaScript Skills: https://developer.mozilla.org/
- Python Programming Proficiency: https://docs.python.org/3/
- Version Control with Git: https://git-scm.com/doc
- Render Deployment Basics: https://render.com/docs

Project Workflow

1. Collect and clean the UNDP HDI dataset; handle missing values via mean imputation.
2. Engineer features — notably log-transforming GNI per capita.
3. Train and compare Linear Regression vs Random Forest; keep the higher-R^2 model.
4. Serialize the model with joblib and export supporting EDA graphs and country data.
5. Build the Flask app: Home, Predict, and History dashboards plus a JSON API.
6. Seed demo prediction history so the History dashboard isn't empty on first run.
7. Test functionality (unit tests) and performance (response-time load testing).
8. Deploy to Render (Gunicorn WSGI server) and verify the live public URL.